

Memoria

-

Ejercicio React



Universidad Pública de Navarra
Nafarroako Unibertsitate Publikoa

Ruben Cameo Saez

1. Introducción

El objetivo de este ejercicio es aplicar y consolidar los conceptos fundamentales de React trabajados en clase, tales como el uso de componentes, el manejo de eventos y la gestión de estados.

Para ello, se ha desarrollado una aplicación que cumple con los siguientes requisitos:

- Creación de dos campos numéricos para los valores A y B.
- Implementación de cuatro botones para las operaciones básicas (+, −, ×, ÷).
- Uso de un componente específico de Resultado que muestra la operación aplicada a los valores A y B en tiempo real.
- Diferenciación estética del botón de operación que se encuentra activo en cada momento para mejorar la experiencia de usuario.

2. Desarrollo y funcionalidad

Con el objetivo de cumplir los requisitos del ejercicio, se ha definido la siguiente estructura de distribución:

```
ejercicio_react_calculadora/  
├── src/  
│   ├── assets/  
│   ├── components/  
│   │   ├── Calculadora.jsx  
│   │   ├── calculadora.css  
│   │   ├── Header.jsx  
│   │   ├── header.css  
│   │   ├── Footer.jsx  
│   │   └── Resultado.jsx  
│   ├── pages/  
│   │   ├── Home.jsx  
│   │   └── home.css  
│   ├── App.jsx  
│   ├── App.css  
│   ├── main.jsx  
│   └── index.css  
└── index.html
```

Fig 1. Estructura de componentes y archivos del proyecto.

Como se observa en el esquema, el punto de entrada principal es App.jsx. Este componente actúa como contenedor global, albergando la estructura base compuesta por el Header, el cuerpo principal (Home) y el Footer.

Dentro de la página principal (Home.jsx), se han integrado los componentes clave solicitados:

- **Calculadora.jsx:** Es el núcleo de la aplicación, donde reside la lógica de los estados para los valores A y B, así como los botones de operación.
- **Resultado.jsx:** Un componente especializado en recibir los datos procesados y renderizar el valor final de forma clara para el usuario.

La vista general de la aplicación es la que se indica a continuación:



Fig 2. Interfaz de usuario final con la indicación de los componentes funcionales

2.1 App.jsx

Este archivo actúa como el componente raíz y el punto de entrada de la aplicación.

Aspectos clave de la implementación:

- Se emplea una estructura que envuelve el contenido entre un Header y un Footer. Gracias a las clases de Bootstrap (d-flex flex-column min-vh-100), se garantiza que el pie de página se mantenga siempre en la parte inferior de la pantalla, independientemente de la cantidad de contenido.
- Se implementa la librería react-router-dom mediante los componentes `<Routes>` y `<Route>`. Aunque actualmente solo cuenta con la ruta principal (`/`), esta estructura permite que la aplicación sea escalable, facilitando la adición de nuevas páginas en el futuro.
- Se ha definido una etiqueta main con la clase `flex-grow-1`. Esto es fundamental para el diseño, ya que obliga al cuerpo de la aplicación (donde se renderiza el componente Home) a ocupar todo el espacio disponible entre la cabecera y el pie de página.

2.2 Home.jsx

Este componente va a actuar como la página principal de la aplicación. Su función es estructural y organizativa, sirviendo de puente entre el App.jsx y el componente funcional, llamado Calculadora.jsx.

Aspectos clave de la implementación:

- Utilizamos un componente `<Container>` con clases de tipo Flexbox (`d-flex`, `align-items-center`, `justify-content-center`) para asegurar que la calculadora aparezca perfectamente centrada tanto vertical como horizontalmente en la pantalla.
- Hemos establecido el principio de responsabilidad única al delegar toda la lógica matemática y de estado al componente `<Calculadora />`, manteniendo el código de la página limpio y fácil de leer.
- Al envolver la calculadora en un sistema de `<Row>` y `<Col>`, se garantiza que el contenido se adapte correctamente al ancho de la pantalla disponible, respetando los márgenes del contenedor de Bootstrap.

2.3. Calculadora.jsx

En este componente es donde he implementado la lógica principal del ejercicio. He utilizado tres estados con el `useState`: uno para el Número A, otro para el Número B y un tercero para la operación seleccionada.

Aspectos clave de la implementación:

- Para los campos de entrada, he usado el evento `onChange`. Es importante destacar que he envuelto el valor en `Number()` para asegurar que React realice operaciones matemáticas reales y no concatene los valores como si fueran texto.
- En lugar de crear funciones complejas, el cálculo se realiza directamente en el cuerpo del componente mediante condicionales (`if/else`). Así, el resultado se actualiza en tiempo real cada vez que el usuario cambia un número o pulsa un botón de operación.
- He creado la función `marcadosignoelegido` para cumplir con el requisito de diferenciar el botón activo. Esta función compara el estado de operación con el símbolo de cada botón. Si coinciden, devuelve la variante de Bootstrap `dark` (botón relleno), y si no, devuelve `outline-dark` (solo el borde).
- Una vez obtenido el cálculo, se lo envió al componente `<Resultado />` mediante la prop `resultado_obtenido`. Esto permite separar la lógica de la visualización.

2.4. Resultado.jsx

Este componente funcional tiene como responsabilidad la representación visual del cálculo final. Se encarga de recibir, procesar y mostrar el valor obtenido de forma aislada.

Aspectos clave de la implementación

- Utiliza la propiedad `props.resultado_obtenido` para renderizar dinámicamente el valor calculado en otros niveles de la aplicación.
- Emplea clases de Bootstrap (`d-flex`, `align-items-center`) para el posicionamiento, junto con estilos en línea (`inline styles`) para definir una estética personalizada: borde sólido, fondo específico y límites de ancho para asegurar la responsividad.
- Al estar exportado de forma independiente, permite ser reutilizado en cualquier sección del desarrollo que requiera mostrar un resultado alfanumérico con este formato.

3. Implementación para móviles

Para garantizar una experiencia adecuada de usuario en dispositivos móviles, se ha seguido una estrategia de diseño responsivo. El objetivo principal ha sido adaptar los elementos de la interfaz para que mantengan su usabilidad sin importar el tamaño de la pantalla.

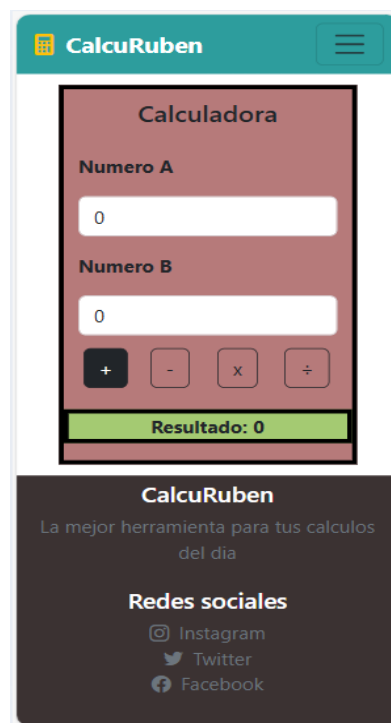


Fig 3. Captura de como se ve el móvil

3.1. Footer.jsx

El componente de pie de página se ha estructurado utilizando un sistema de rejilla (grid) flexible. La disposición de los elementos se adapta dinámicamente según el ancho de la pantalla del usuario, garantizando que el contenido sea siempre legible.

Para lograr este comportamiento, se ha configurado la división de las columnas de la siguiente manera:

- **xs={12}**: En resoluciones pequeñas (móviles), cada columna ocupa el 100% del ancho del contenedor (las 12 columnas del grid). Esto permite que los bloques de información se apilen verticalmente, evitando el desbordamiento horizontal y facilitando la lectura táctil.
- **md={4}**: En resoluciones medias y superiores (tablets y ordenadores), el espacio se redistribuye para que cada bloque ocupe solo un tercio del ancho total (4 de las 12 columnas).

3.2. Calculadora.jsx

La capacidad de respuesta de la interfaz recae en el uso estratégico de los atributos xs y sm dentro de los componentes Col. Esto permite que la distribución cambie radicalmente según el ancho de pantalla detectado:

- Se ha asignado un valor de 12 a las columnas de los campos de entrada (Número A y Número B). Esto garantiza que en dispositivos móviles (pantallas extra pequeñas), cada elemento ocupe el 100% del ancho disponible, forzando un diseño vertical que es mucho más cómodo para el uso con una sola mano.
- A diferencia de los inputs, los botones de operación utilizan un valor de 3. Esto permite que, incluso en móviles, los cuatro botones queden alineados en una sola fila ($3 \times 4 = 12$), ahorrando espacio vertical y manteniendo las herramientas de cálculo siempre a la vista.
- Al superar el breakpoint de pantallas pequeñas (sm), las columnas reducen su ancho relativo. Esto hace que las etiquetas y los inputs dejen de estar uno debajo del otro para pasar a una disposición horizontal, aprovechando mejor el espacio lateral en tablets o monitores.

3.3. Resultado.jsx

Aunque no depende directamente del sistema de rejilla, este componente asegura su responsividad mediante propiedades CSS:

- El uso de width: 100% en combinación con maxWidth: 250px permite que el cuadro de resultado se expanda en móviles para ser claramente visible, pero mantenga dimensiones proporcionadas en pantallas más grandes.